



PHYSICS IS FUN!

Devis Rossini



WHO AM I?

Devis Rossini
R&D and Physics Programmer @ Milestone



devis.rossini@gmail.com



linkedin.com/in/devis-rossini/



[@devisrosso](https://www.instagram.com/devisrosso)

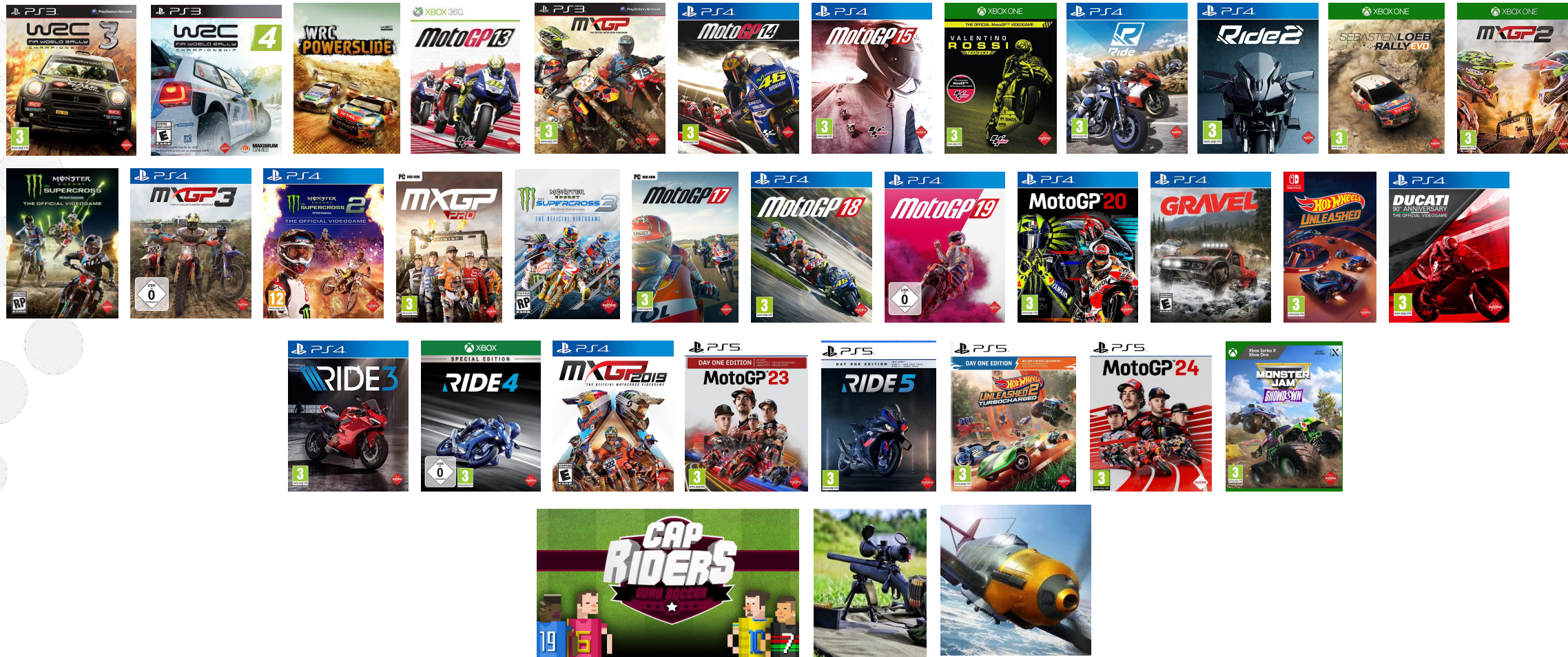


[devis.rossini#6634](https://discord.com/users/devis.rossini#6634)



WHO AM I?

"It's not who I am underneath, but what I do that defines me" (Bruce Wayne)





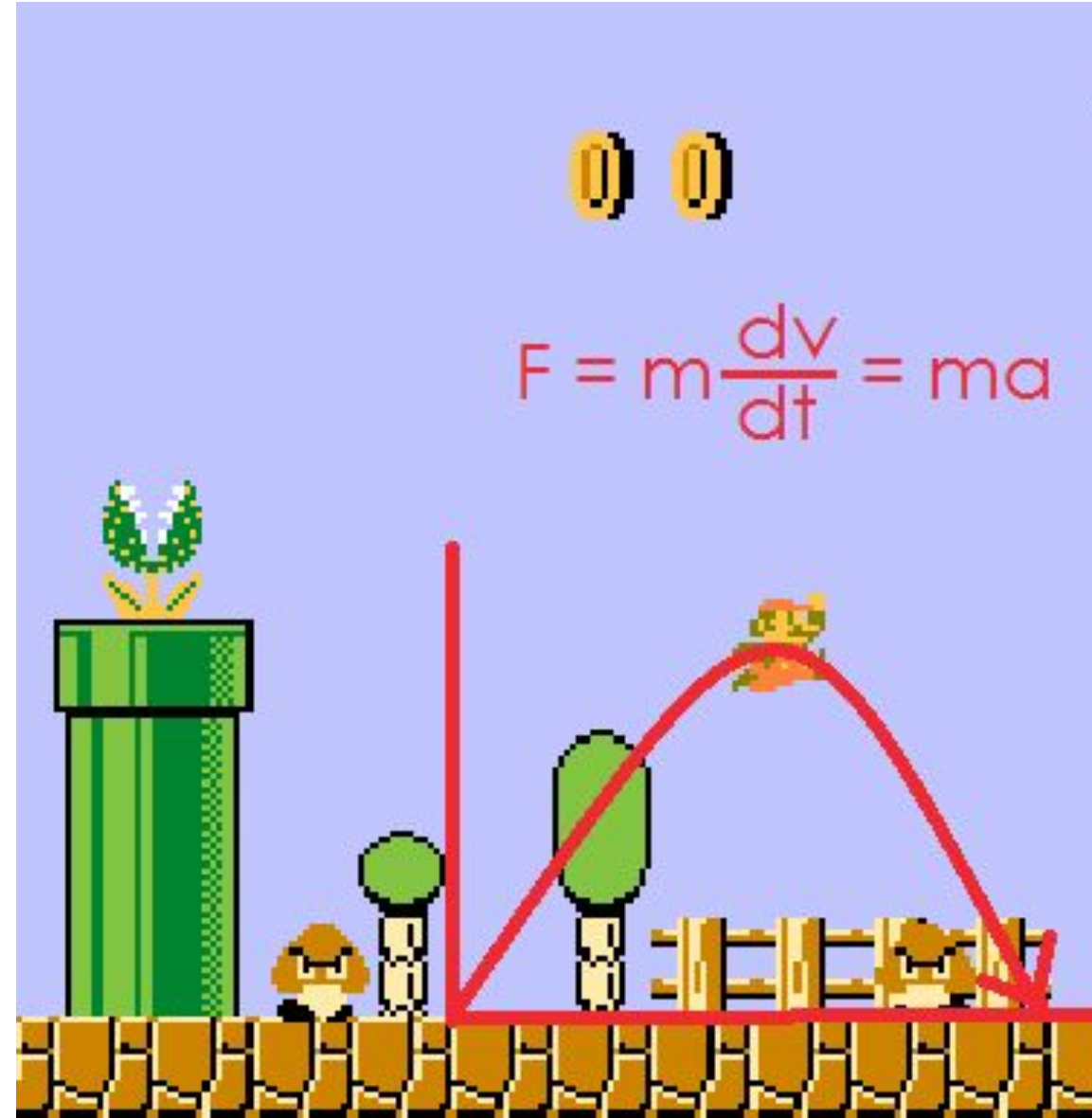
LESSON PLAN

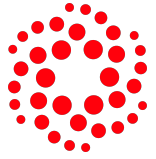
What will we do together?



LESSON PLAN

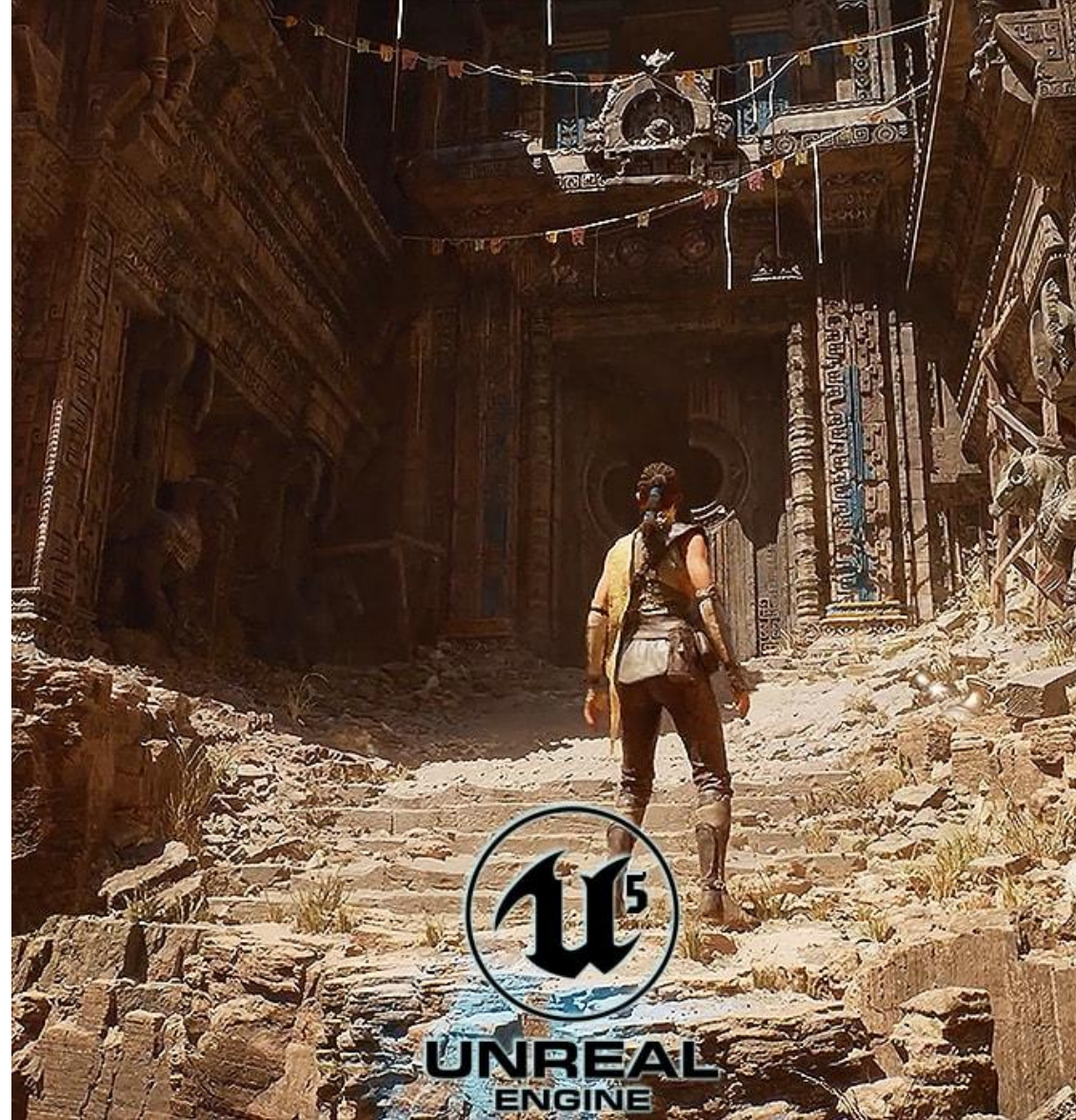
- Physics Simulation
 - Definition
 - The Equations of Motion
 - Numerical Integration
 - Spring Systems
 - Rigid Body Integration
 - Delta time: Variable vs Fixed

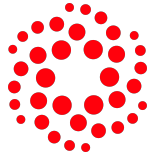




LESSON PLAN

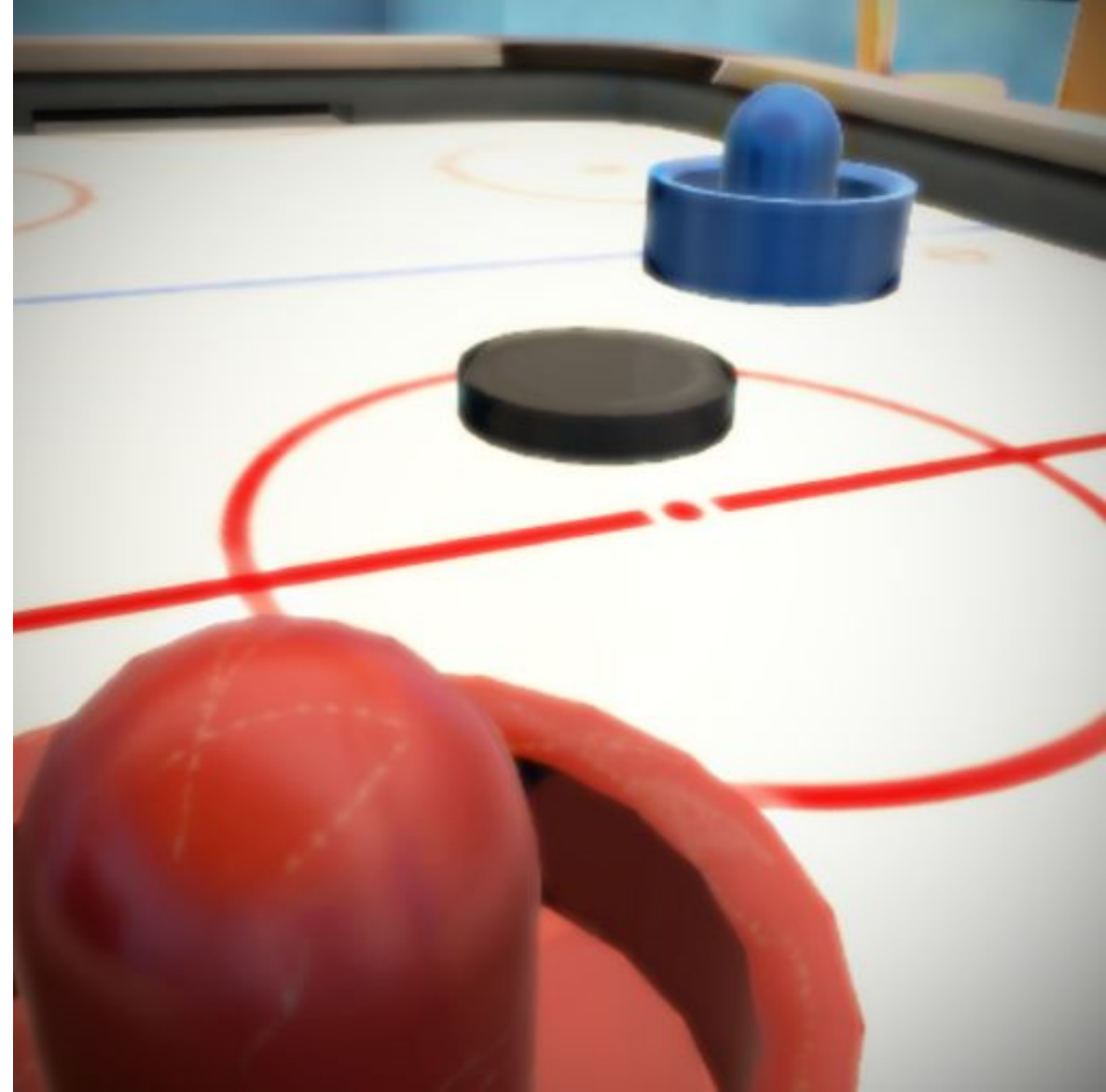
- Physics in Unreal Engine
 - Base Concepts
 - BodyInstance
 - Physics-based Movement
 - Collisions and Physics-based Interactions
 - Raycasts and Sweep Tests





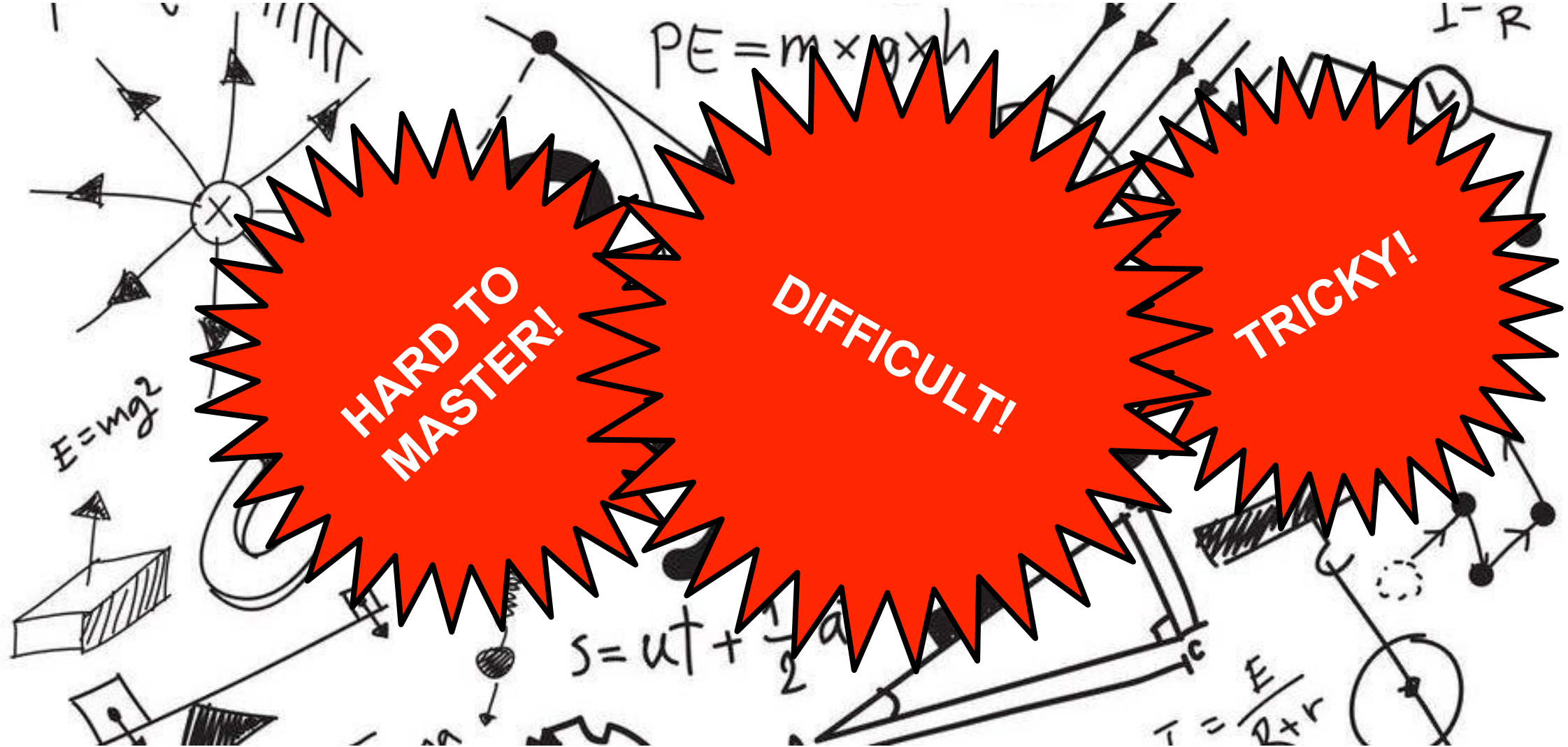
LESSON PLAN

- Let's build a (simple) air hockey game!





BEFORE STARTING

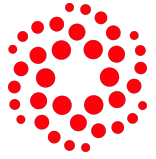


2008



1999





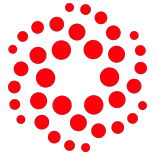
BEFORE STARTING



“As long as it works”



...is the root of all evils.



BEFORE STARTING



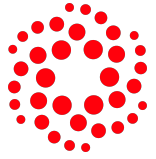
Realism is not (necessarily)
fun!

Physics Sim is not
(necessarily) realistic.



PHYSICS SIMULATION

What is it? How does it work?



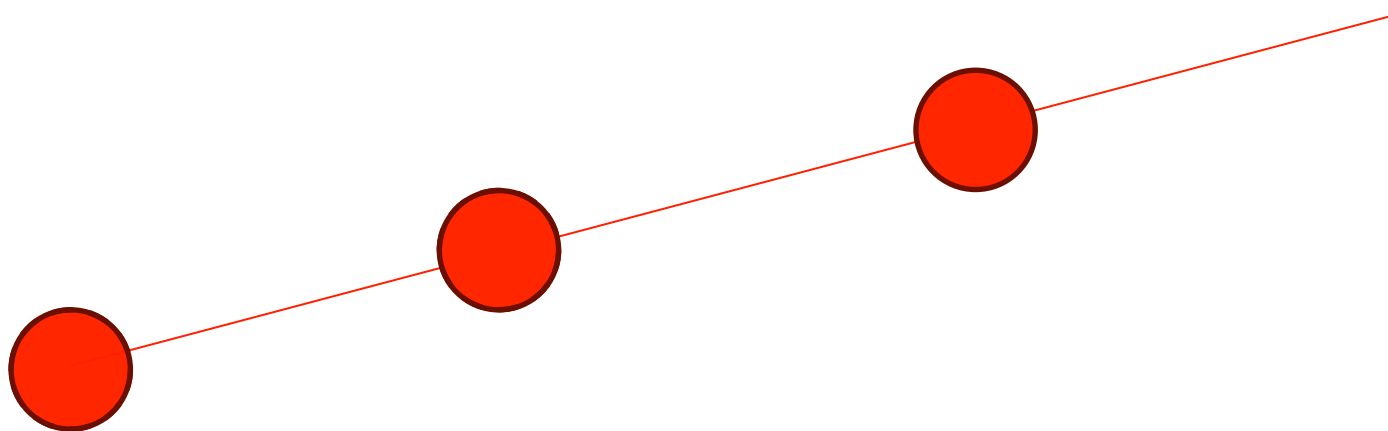
BASE CONCEPTS

A physics simulation works by making many small predictions based on the laws of physics.

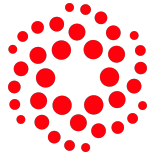


BASE CONCEPTS

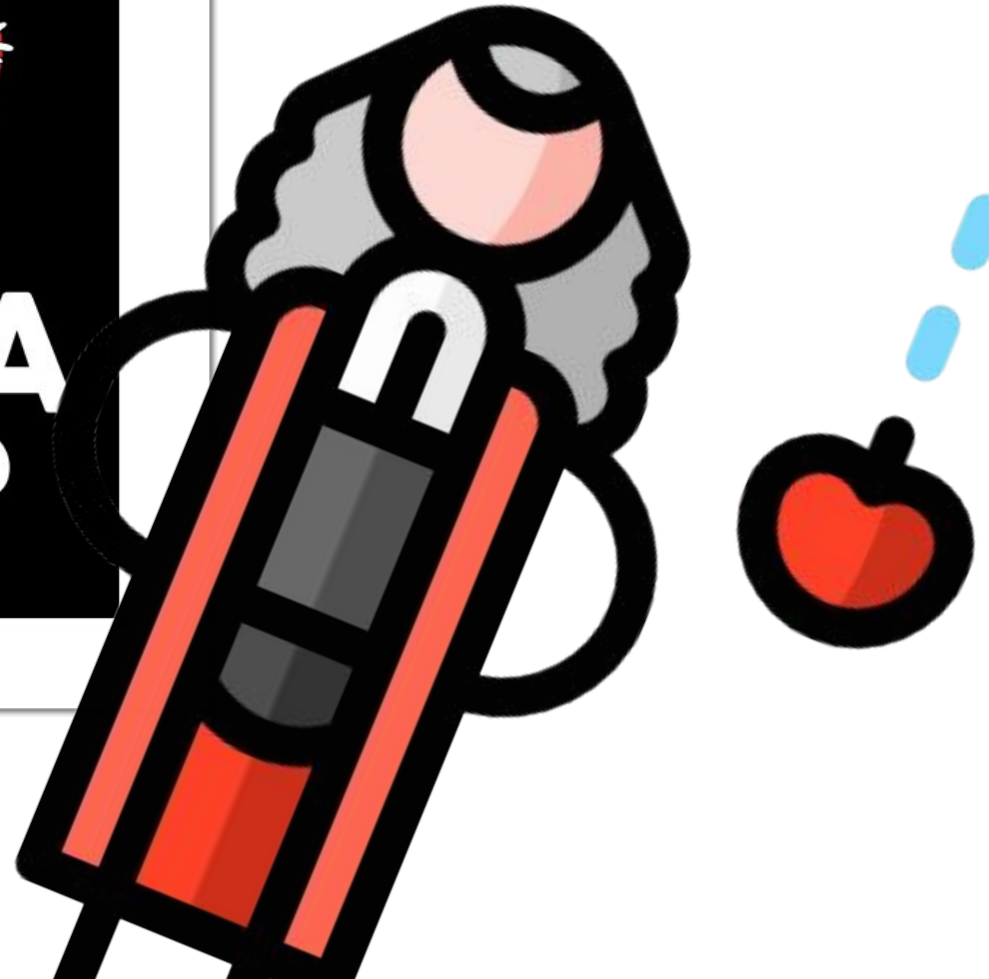
“The object is here, and is traveling this fast in that direction, so in a short amount of time it should be over there”

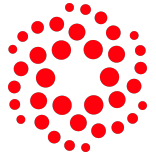


We perform these predictions using a mathematical technique called **integration**



BASE CONCEPTS





THE EQUATIONS OF MOTION

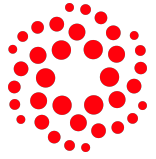
$$F = ma \quad a = \frac{F}{m}$$

Acceleration is the rate of change in velocity over time:

$$\frac{dv}{dt} = a = \frac{F}{m}$$

Similarly, velocity is the rate of change in position over time:

$$\frac{dx}{dt} = v$$



THE EQUATIONS OF MOTION

If we know the current position and velocity of an object, and the forces that will be applied to it, we can integrate to find its position and velocity at some point in the future



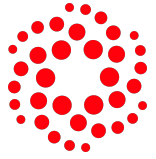
THE EQUATIONS OF MOTION





THE EQUATIONS OF MOTION

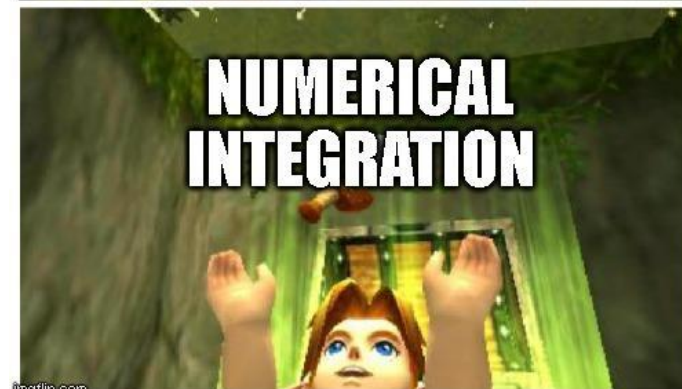




NUMERICAL INTEGRATION

First, start at an **initial position and velocity**, then **take a small step forward** to find the velocity and position at a future time.

Then repeat this, moving forward in small time steps, using the result of the previous calculation as the starting point for the next.





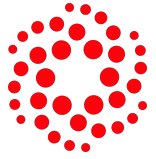
NUMERICAL INTEGRATION

Let's call our current time **t**, and the time step **dt** or '**delta time**'.

`acceleration = force / mass`

`change in position = velocity * dt`

`change in velocity = acceleration * dt`



NUMERICAL INTEGRATION

This makes intuitive sense because if you're in a car traveling 60 kilometers per-hour, in one hour you'll be 60 kilometers further down the road.

Similarly, a car accelerating 10 kilometers per-hour-per-second will be moving 100 kilometers per-hour faster after 10 seconds.





NUMERICAL INTEGRATION

This logic only holds when acceleration and velocity are constant.

But even when they're not, it's still a pretty decent approximation to start with.



NUMERICAL INTEGRATION

```
double t = 0.0;
float dt = 1.0f;
float velocity = 0.0f;
float position = 0.0f;
float force = 10.0f;
float mass = 1.0f;

while ( t <= 10.0 )
{
    position = position + velocity * dt;
    velocity = velocity + ( force / mass ) * dt;
    t += dt;
}
```

t=0:	position = 0	velocity = 0
t=1:	position = 0	velocity = 10
t=2:	position = 10	velocity = 20
t=3:	position = 30	velocity = 30
t=4:	position = 60	velocity = 40
t=5:	position = 100	velocity = 50
t=6:	position = 150	velocity = 60
t=7:	position = 210	velocity = 70
t=8:	position = 280	velocity = 80
t=9:	position = 360	velocity = 90
t=10:	position = 450	velocity = 100

What we just did is a type of integration called **explicit Euler**.



NUMERICAL INTEGRATION

Euler integration is the most basic numerical integration technique. It is only 100% accurate when the rate of change is constant over the timestep.

Since acceleration is constant in the example above, the integration of velocity is without error. However, we are also integrating velocity to get position, and velocity is increasing due to acceleration. This means there is error in the integrated position.

Just how large is this error? Let's find out!



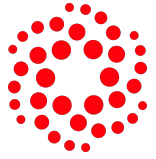
NUMERICAL INTEGRATION

$$S = s(t_0) + v(t_0)t + \frac{1}{2}a(t - t_0)^2$$



...Let's write some code!





NUMERICAL INTEGRATION

In our example, we have:

$$\begin{aligned}t_0 &= 0 \\s(t_0) &= 0 \\v(t_0) &= 0 \\t &= 10s \\a &= 10 \frac{m}{s^2}\end{aligned}$$

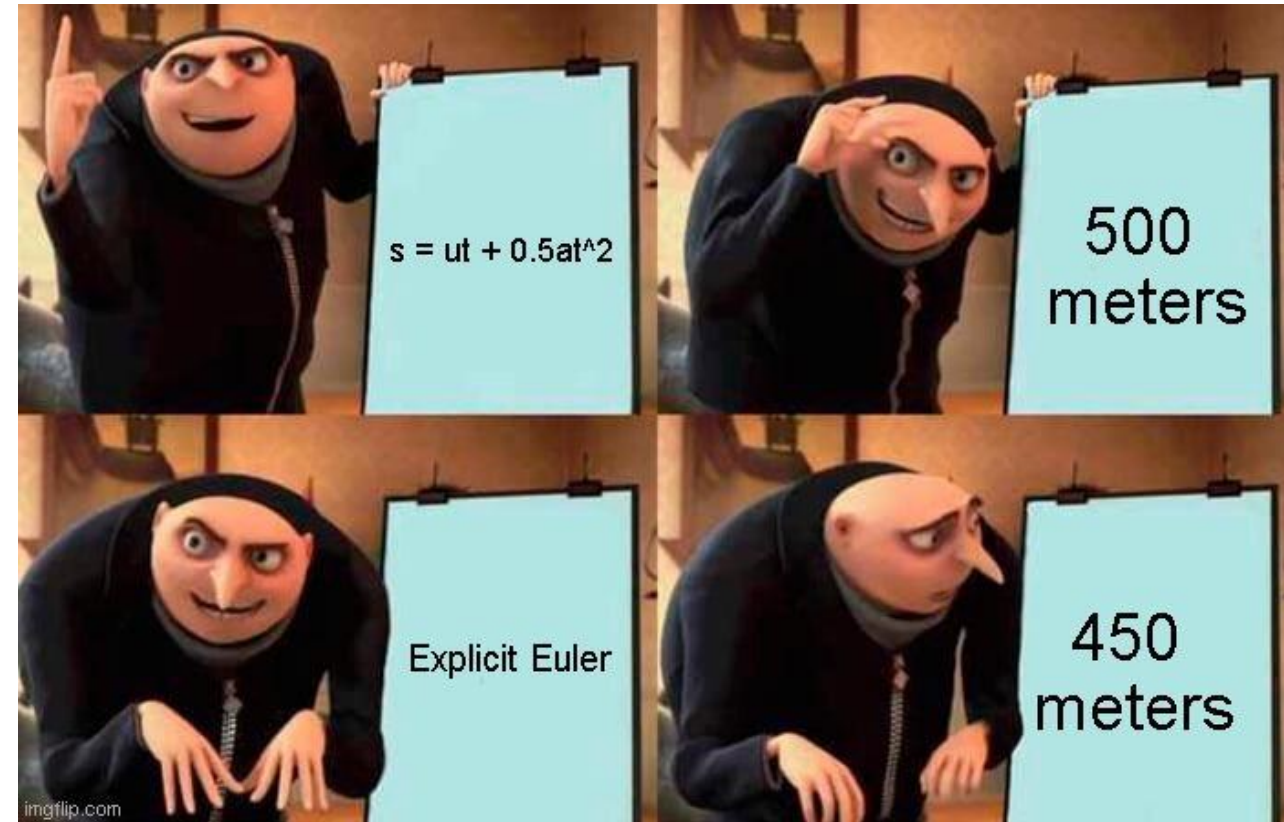
$$\begin{aligned}s &= 0 + 0 \cdot t + 0.5at^2 \\s &= 0.5(10)(10^2) \\s &= 0.5(10)(100) \\s &= \mathbf{500 \text{ meters}}\end{aligned}$$



NUMERICAL INTEGRATION

After 10 seconds, the object should have moved 500 meters, but explicit Euler gives a result of 450 meters.

...not so good 😞





NUMERICAL INTEGRATION

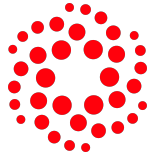
But don't panic!

A delta time of 1 second is unrealistic for a game physics simulation. Physics usually steps with a **delta time closer to the display framerate**.

With $dt = 1/100$:

t=9.90:	position = 489.552155	velocity = 98.999062
t=9.91:	position = 490.542145	velocity = 99.099060
t=9.92:	position = 491.533142	velocity = 99.199059
t=9.93:	position = 492.525146	velocity = 99.299057
t=9.94:	position = 493.518127	velocity = 99.399055
t=9.95:	position = 494.512115	velocity = 99.499054
t=9.96:	position = 495.507111	velocity = 99.599052
t=9.97:	position = 496.503113	velocity = 99.699051
t=9.98:	position = 497.500092	velocity = 99.799049
t=9.99:	position = 498.498077	velocity = 99.899048
t=10.00:	position = 499.497070	velocity = 99.999046

Not so bad 😊



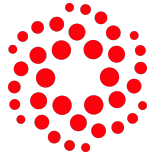
NUMERICAL INTEGRATION

Using a small timestep, explicit euler gives good results...

...but what about the case where acceleration isn't constant?

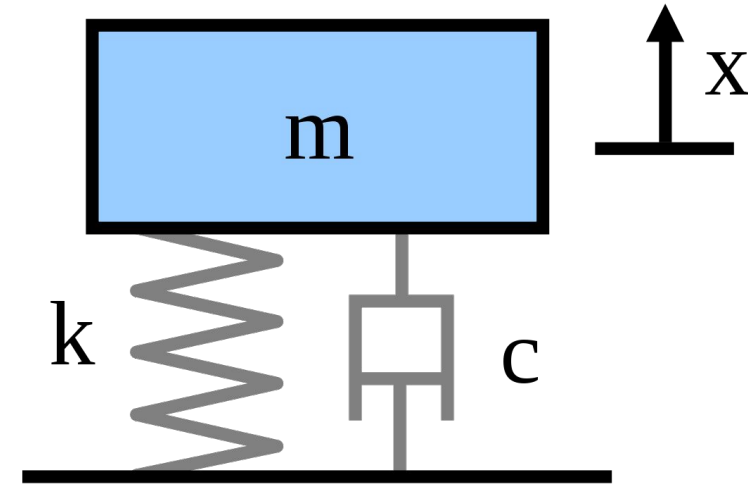
Let's talk about **springs** and **harmonic oscillator**!





SPRING SYSTEMS

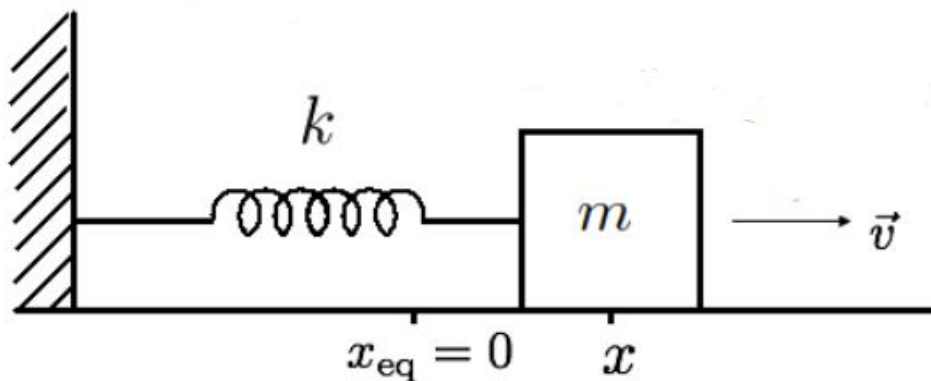
A Damped Harmonic Oscillator consist of a mass attached to a spring and its motion is damped by some kind of friction.



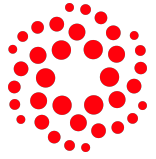


SPRING SYSTEMS

There is a **force proportional to the distance of the object that pulls it towards the origin**, and a **force proportional to the velocity of the object, but in the opposite direction**, which slows it down.



$$F = -kx - c \frac{dx}{dt} = -kx - cv$$



SPRING SYSTEMS

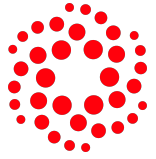
Example:

Mass: $m = 1 \text{ kg}$

Initial position: $x = 1000 \text{ mt}$ (assume neutral position as the origin)

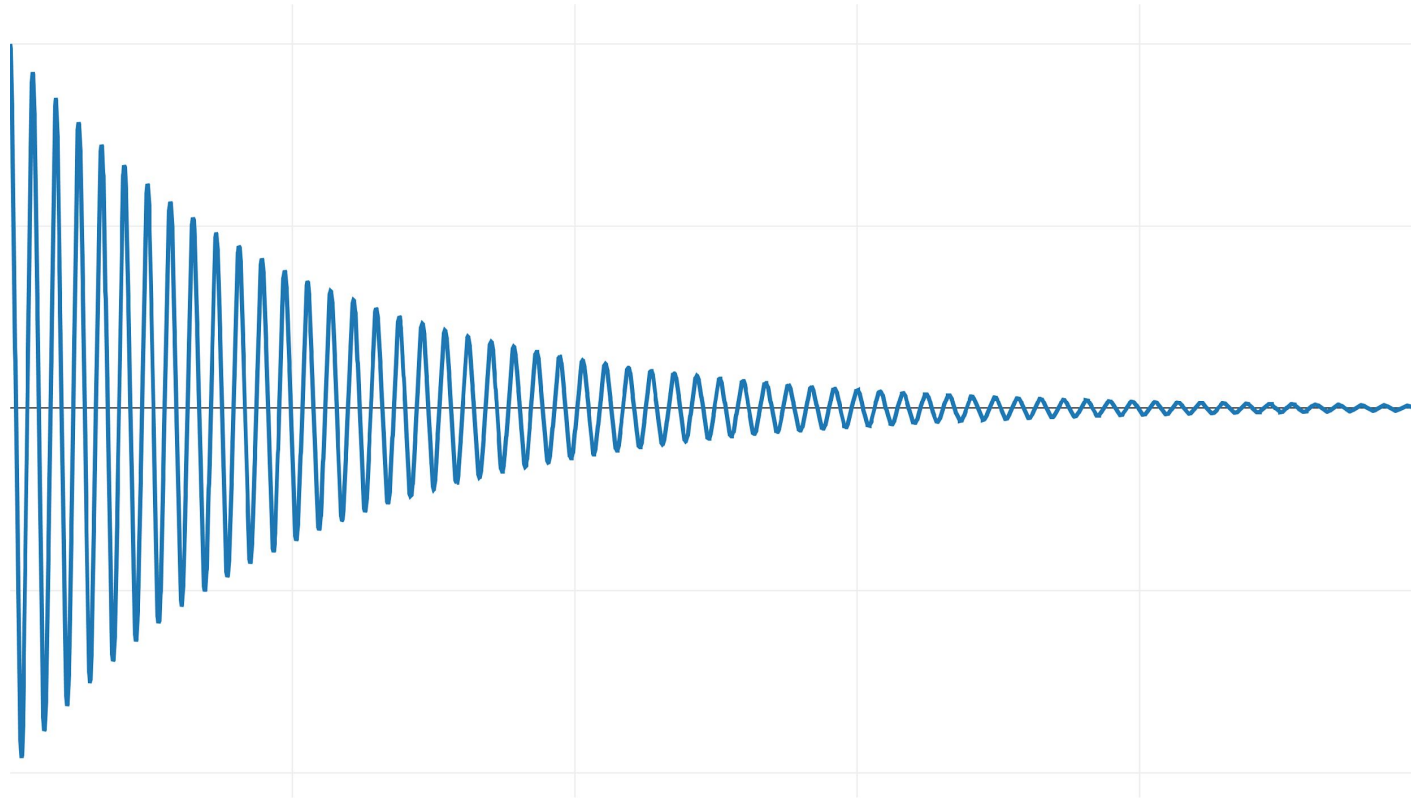
Spring coefficient: $k = 15$

Damping coefficient: $c = 0.1$



SPRING SYSTEMS

We should have something like:



imgflip.com

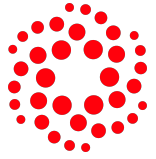
CHALLENGE ACCEPTED





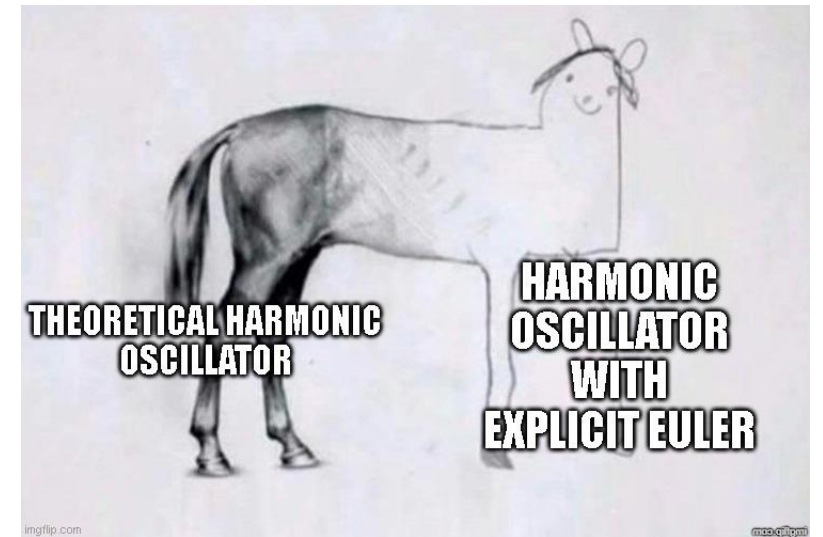
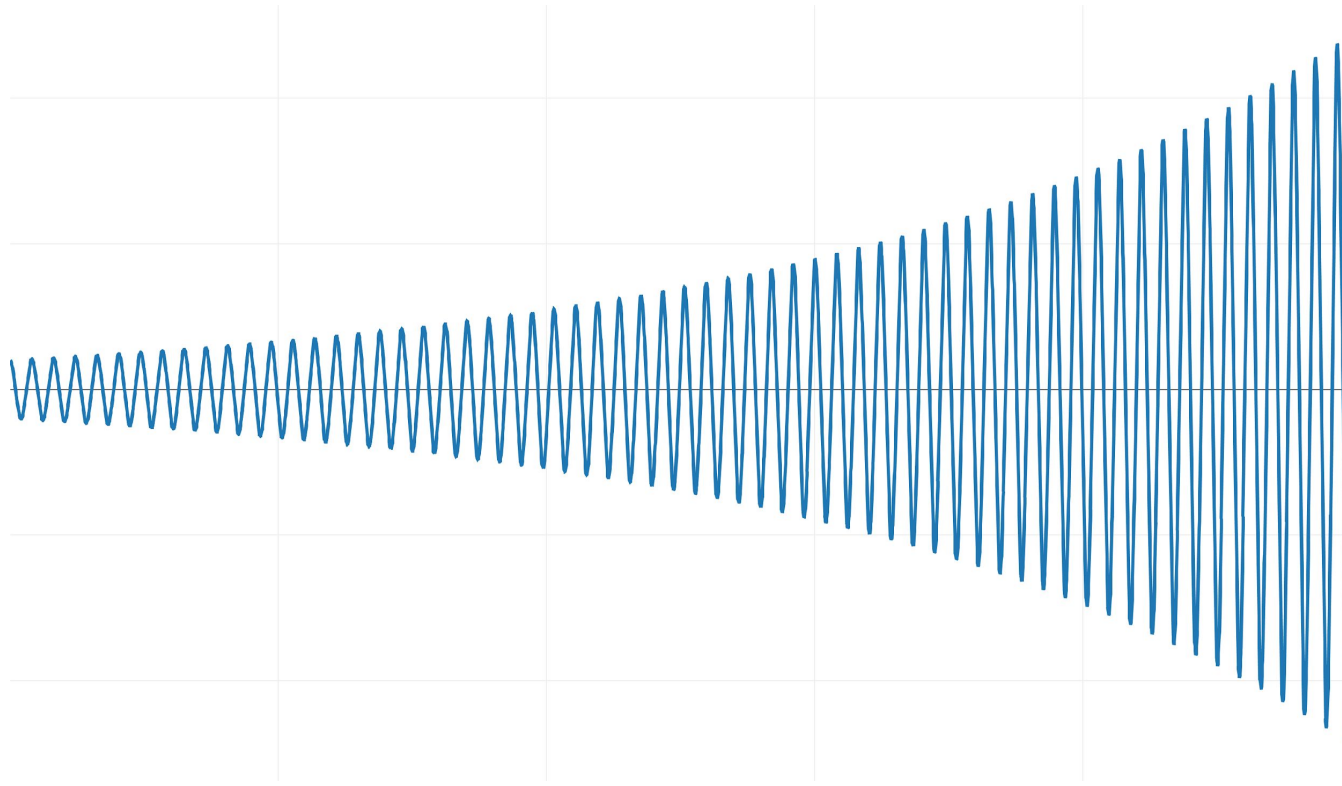
...Let's write some code!





SPRING SYSTEMS

If we apply explicit Euler, it yields a different result, even with $dt = 1/100$:





IMPLICIT EULER

We can solve considering a different integrator, **semi-implicit Euler**.

Switching from explicit to semi-implicit Euler is as simple as changing:

$$\begin{aligned}\mathbf{position} &= \mathbf{position} + \mathbf{velocity} * dt \\ \mathbf{velocity} &= \mathbf{velocity} + \mathbf{acceleration} * dt\end{aligned}$$

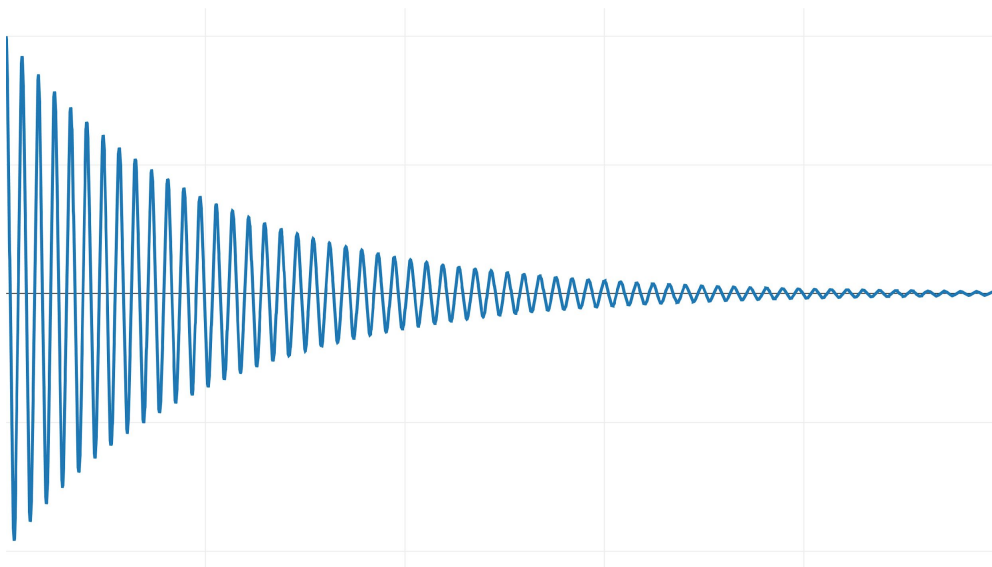
to:

$$\begin{aligned}\mathbf{velocity} &= \mathbf{velocity} + \mathbf{acceleration} * dt \\ \mathbf{position} &= \mathbf{position} + \mathbf{velocity} * dt\end{aligned}$$

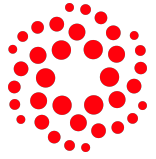


IMPLICIT EULER

Using a semi-implicit Euler integrator with $dt = 1/100$ yields a stable result, very close to the theoretical solution:



Why? The theory behind this result is quite complex. This happens because implicit euler has the properties of a **symplectic integrator**.



OTHER INTEGRATION METHODS

- **Verlet Integration**
 - Greater accuracy than implicit Euler (2nd order)
 - Less memory usage when simulating large number of objects
 - Symplectic
- **Runge Kutta Methods** (in particular, RK4)
 - 4th order integrator, extreme accuracy

...so, they are better, isn't it? Not necessarily, because they are complex and (under specific circumstances) they lose energy.

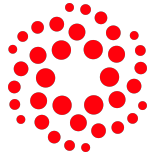


RIGID BODY INTEGRATION

Rigid bodies are objects that cannot bend, compress or deform in any way. To simulate the motion of rigid bodies, we must analyze:

- **Kinematics:** is the study of how an object moves in the absence of forces
- **Dynamics:** how an object reacts to forces

We will see how to integrate vector quantities, handle rotations and integrate them.



RIGID BODY INTEGRATION

We want our object to be able to move in three dimensions: left and right, forward and back, up and down. If we apply the equations of motion to each dimension separately, we can integrate each dimension in turn to find the motion of the object in three dimensions.

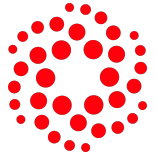
Or... we could just use vectors.



Equations
of motion
for x , y ,
 z separately



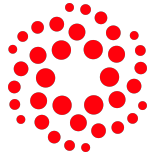
Vectors



RIGID BODY INTEGRATION

In C++ we implement vectors using a struct as follows:

```
struct Vector
{
    float X;
    float Y;
    float Z;
};
```



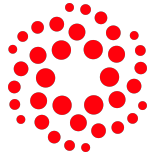
RIGID BODY INTEGRATION

Addition of two vectors is defined as adding each component together.

Multiplying a vector by a floating-point number is the same as just multiplying each component.

```
Vector operator+(const Vector &Other)
{
    Vector Result;
    Result.X = X + Other.X;
    Result.Y = Y + Other.Y;
    Result.Z = Z + Other.Z;
    return Result;
}
```

```
Vector operator*(float Scalar)
{
    Vector Result;
    Result.X = X * Scalar;
    Result.Y = Y * Scalar;
    Result.Z = Z * Scalar;
    return Result;
}
```

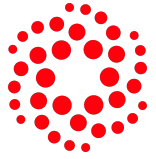


RIGID BODY INTEGRATION

We convert our position, velocity, acceleration and force to vector quantities, then integrate the vectors directly using the equations of motion.

$$\vec{F} = m\vec{a} \qquad \frac{d\vec{v}}{dt} = \vec{a} \qquad \frac{d\vec{x}}{dt} = \vec{v}$$

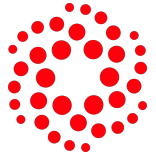
Now that we have the equations of motion in vector form, how do we integrate them? Easy, **exactly the same as we integrated single values.**



RIGID BODY INTEGRATION

```
velocity = velocity + acceleration * dt  
position = position + velocity * dt
```

```
position.X = position.X + velocity.X * dt  
position.Y = position.Y + velocity.Y * dt  
position.Z = position.Z + velocity.Z * dt
```



RIGID BODY INTEGRATION

So, now we can simulate the translation of a rigid body in 3D space...

...but it cannot rotate yet.

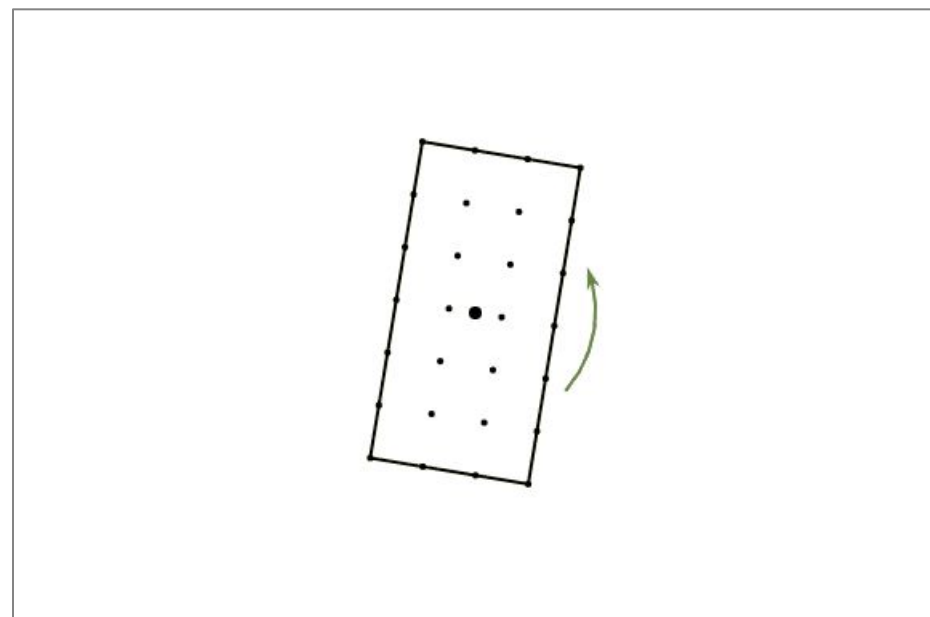
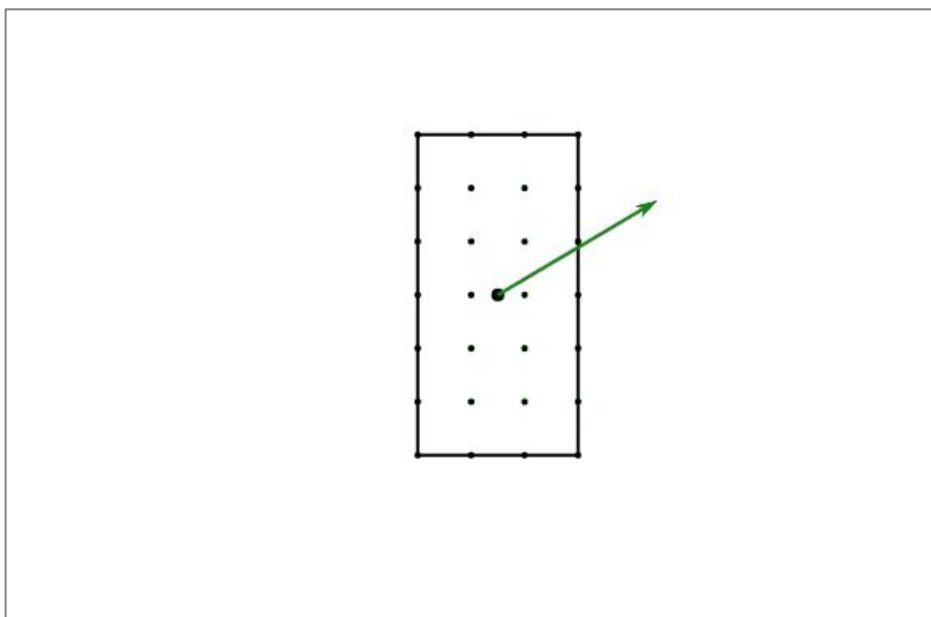
How does a rigid body rotate?

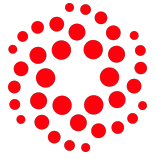




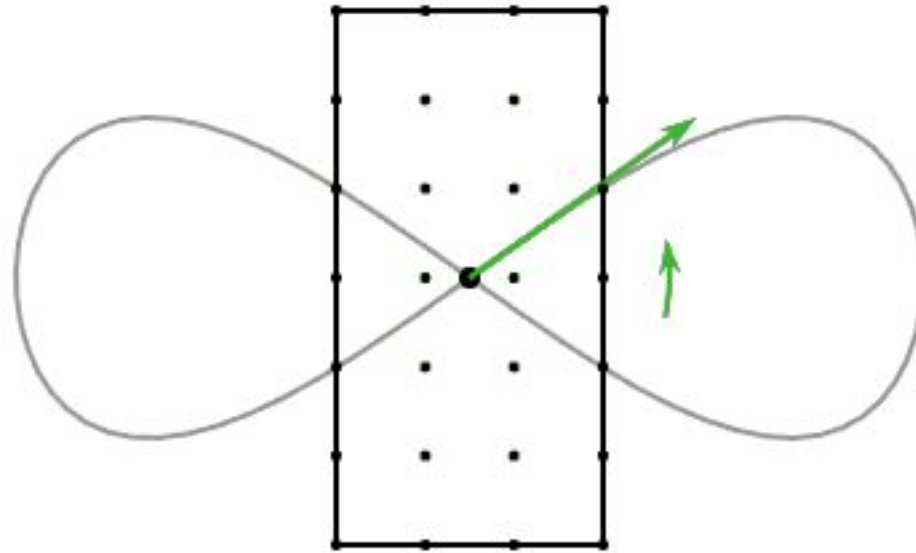
RIGID BODY INTEGRATION

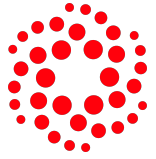
A rigid body cannot deform, so we can treat the linear and rotational parts of an object's motion as being **entirely separate**: a **linear component** (position and velocity) and a **rotational component**, rotating about the center of mass.





RIGID BODY INTEGRATION





RIGID BODY INTEGRATION

How do we represent how the object is rotating?

A rigid body rotation can only ever be around a single axis, so the first thing we need to know is **what that axis is**. We can represent this axis with a unit length vector. Next, we need to know **how fast the object is rotating** about this axis in radians per second.

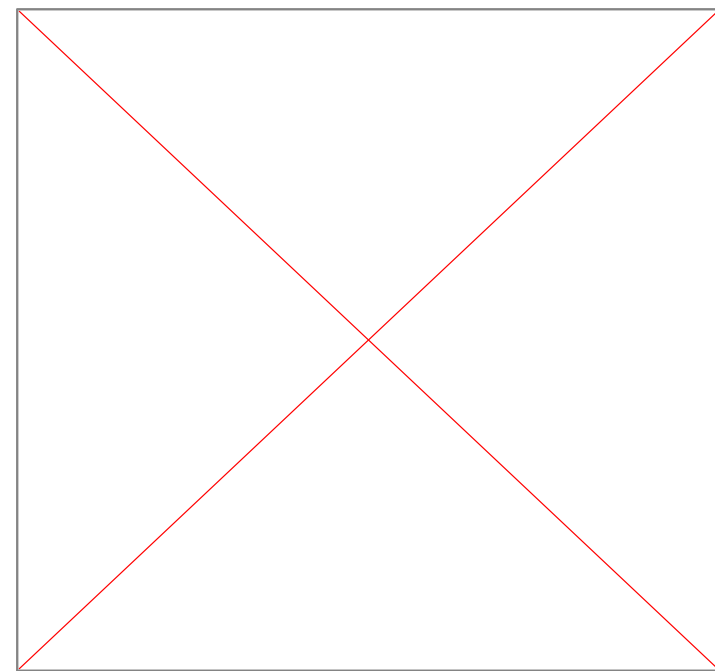
If we know the center of mass of the object, the axis of rotation, and the speed of rotation then we have all the information we need to describe how it is rotating.

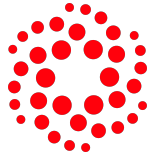


ANGULAR VELOCITY

The standard way of representing rotation over time is by combining the axis and the speed of rotation into a single vector called **angular velocity**.

The length of the angular velocity vector is the speed of rotation in radians while the direction of the vector indicates the axis of rotation.

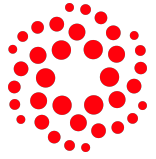




ROTATIONAL INERTIA

Linear velocity and linear momentum are related by mass in linear motion, angular velocity and angular momentum are related by a quantity called the **rotational inertia**. This is a measurement of how much effort it takes to spin an object around an axis. It depends on both the shape of the object and how much it weighs.

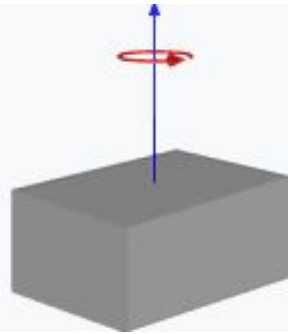
In the general case, rotational inertia is represented by a 3x3 matrix called an **inertia tensor**.



ROTATIONAL INERTIA

For example, if we have a cuboid:

Solid cuboid of width w , height h , depth d , and mass m



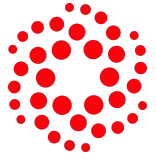
$$I = \begin{bmatrix} \frac{1}{12}m(h^2 + d^2) & 0 & 0 \\ 0 & \frac{1}{12}m(w^2 + h^2) & 0 \\ 0 & 0 & \frac{1}{12}m(w^2 + d^2) \end{bmatrix}$$

https://en.wikipedia.org/wiki/List_of_moments_of_inertia



...Let's write some code!

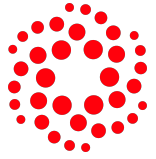




TORQUE

Just as we integrate linear momentum from force, we integrate angular momentum directly from the rotational equivalent of force called **torque**.

You can think of torque just like a force, except that when it is applied it induces a rotation around an axis in the direction of torque vector rather than accelerating the object linearly.



TORQUE

After integrating the angular momentum, we multiply it by the inverse of the rotational inertia to get the angular velocity, and using this angular velocity we integrate to get the rotational equivalent of position called orientation.

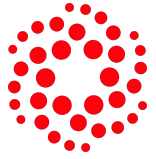
...but wait, it's a bit more complicated :/ It's not easy to represent orientations in 3D.



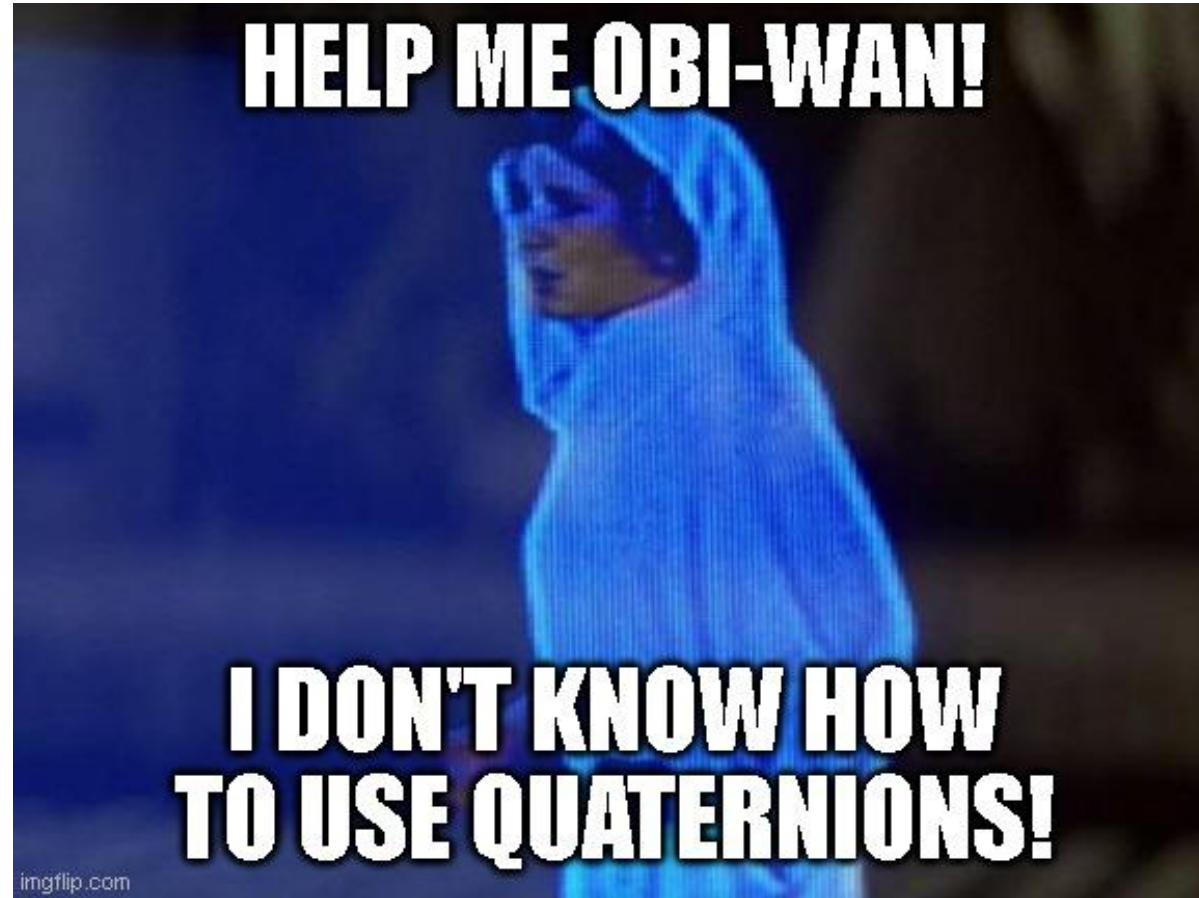
ORIENTATIONS

In two dimensions orientations are easy, you just keep track of angle.

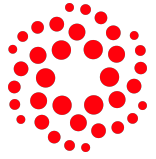
In 3D you must either use a 3x3 rotation matrix or a quaternion to correctly represent the orientation of an object.



QUATERNIONS



<https://www.sjbrown.co.uk/posts/representing-rotations-in-quaternion-arithmetic/>

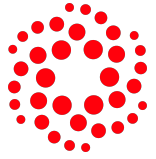


QUATERNIONS



<https://www.gdcvault.com/play/1017653/Math-for-Game-Programmers-Understanding>

<https://www.gdcvault.com/play/1029233/Math-in-Game-Development-Summit>



ORIENTATIONS

(Unit) Quaternions represent an **axis of rotation** and an **amount of rotation about that axis**. This may seem like our angular velocity, but quaternions are four dimensional vectors instead of three – so the math is different.

```
struct Quaternion
{
    float W;
    float X;
    float Y;
    float Z;
};
```



ORIENTATIONS

For a rotation of ϕ about a unit axis $\mathbf{a}$ we will define the representing quaternion $\mathbf{q}$ to be:

$$\mathbf{q} = \left[\cos \frac{\phi}{2}, \mathbf{a} \sin \frac{\phi}{2} \right]$$

So we have: $q_x = a_x \sin \frac{\phi}{2}$ $q_y = a_y \sin \frac{\phi}{2}$ $q_z = a_z \sin \frac{\phi}{2}$ $q_w = \cos \frac{\phi}{2}$

Vice versa, given a quaternion $\mathbf{q} = [q_x, q_y, q_z, q_w]$, it describes a rotation of:

$$\phi = 2 \cos^{-1} q_w$$

around the axis $\mathbf{a} = [a_x, a_y, a_z] = [q_x, q_y, q_z] \cdot \frac{1}{\sin \frac{\phi}{2}}$



ORIENTATIONS

Orientation is a quaternion but angular velocity is a vector. How can we integrate orientation from angular velocity when the two quantities are in different mathematical forms?

The solution is described [here](#) and it's called “**closed form**”:

$$\frac{dq}{dt} = 0.5wq$$

q is the current orientation quaternion

w is the current angular velocity in quaternion form (0,x,y,z)



ORIENTATIONS

Integrating a quaternion, just like integrating a vector, is as simple as doing the integration for each value separately.

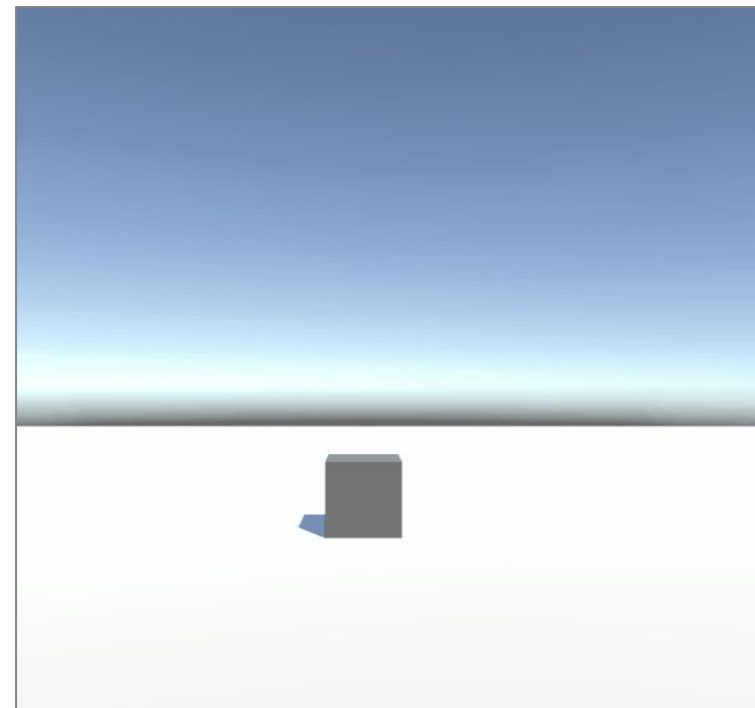
The only difference is that after integrating orientation we must renormalize the orientation quaternion to make it unit length, to ensure that it still represents a rotation.



COMBINED MOTION

Now that we can integrate linear and rotational effects, how can they be combined into one simulation?

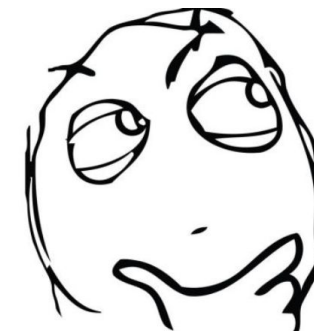
The answer is to just integrate the linear and rotational physics state separately and everything works out.





FORCES AND TORQUE

How can we break down a force applied to a point on the object into a linear force (that changes the linear momentum) and an angular force (that changes the angular momentum)?

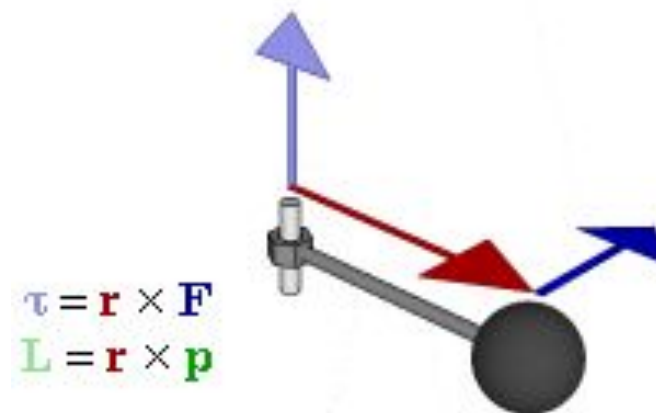


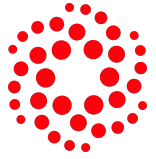
$$\mathbf{T} = \mathbf{F} \times (\mathbf{P} - \mathbf{C})$$

F: force applied

P: force application point

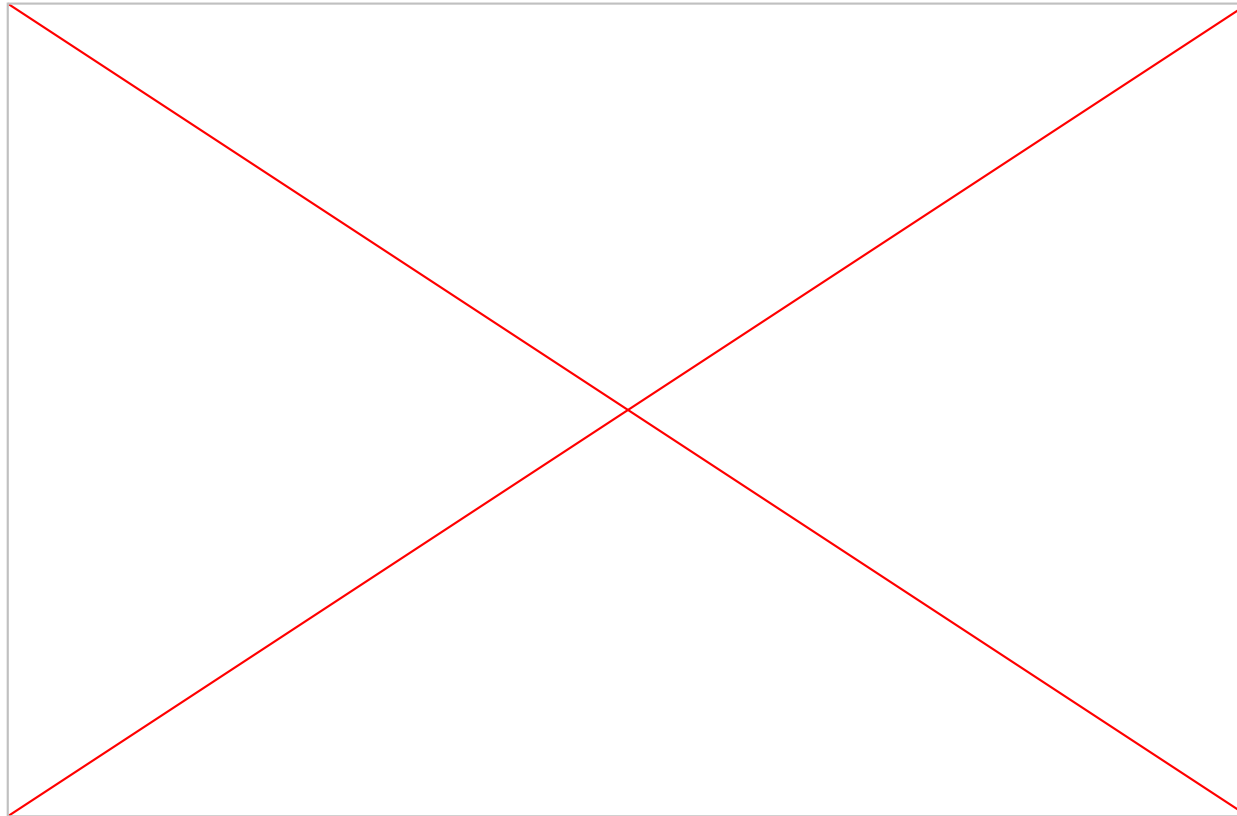
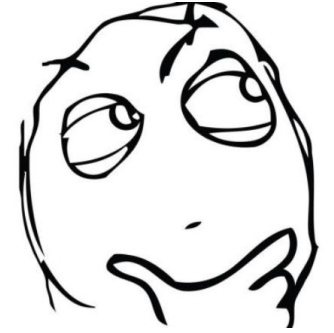
C: center of mass of the object





VELOCITY AT POINT

Do P and Q have the same velocity?



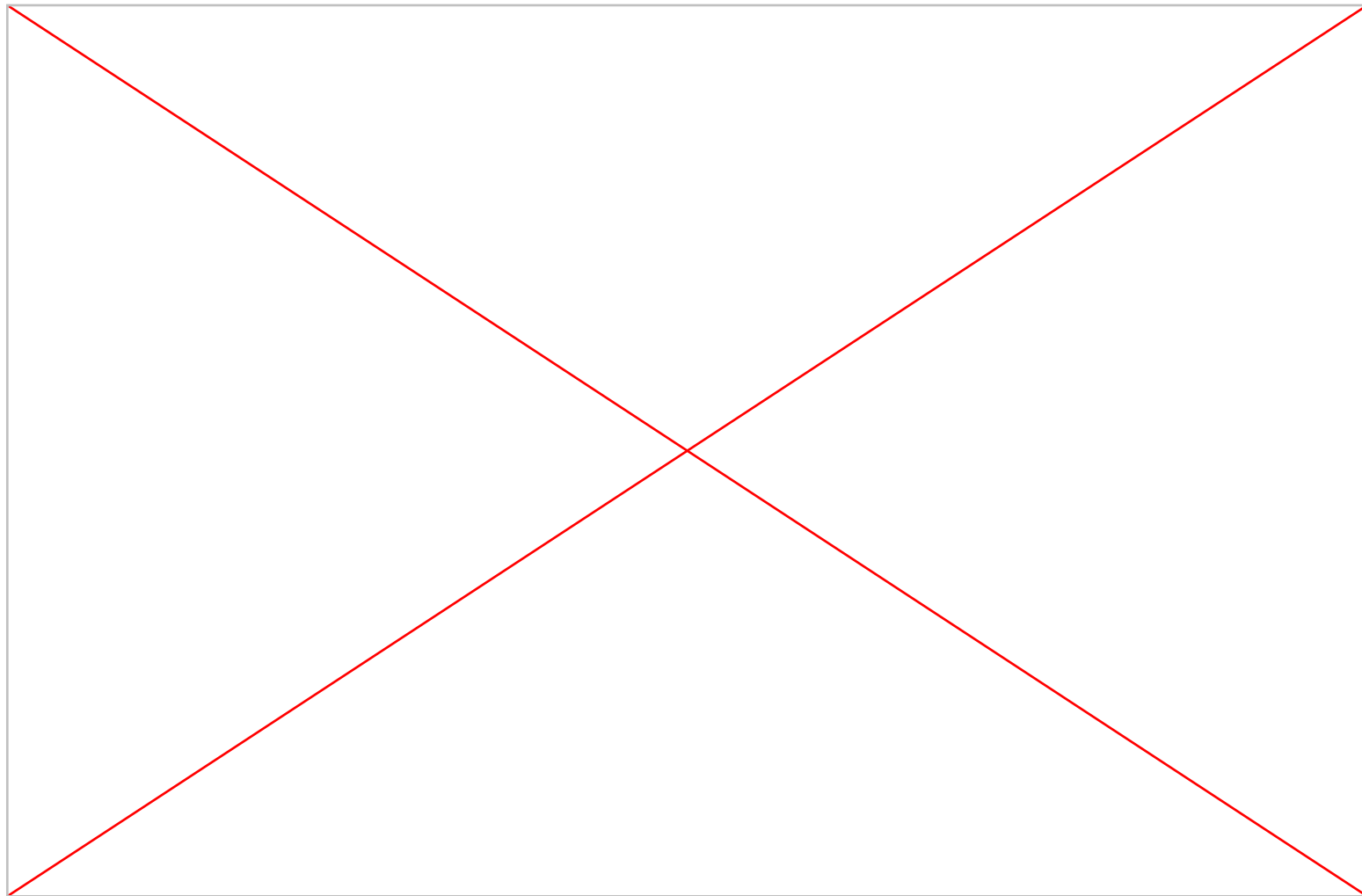


VELOCITY AT POINT

No!

Combining the linear and angular velocities, the total velocity of a point in the rigid body is:

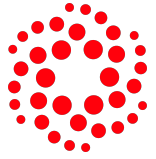
$$\mathbf{v}_Q = \mathbf{v}_P + \mathbf{w} \times (\mathbf{Q} - \mathbf{P})$$





...Let's write some code!





DELTA TIME: VARIABLE vs FIXED

Integration is a technique to advance your physics simulation by some small amount of time called “delta time” (or dt).

How to choose this delta time value?



FIXED DELTA TIME

The simplest way to step forward is with fixed delta time (e.g. 1/60):

```
double t = 0.0;
double dt = 1.0 / 60.0;

while ( !quit )
{
    integrate( state, t, dt );
    render( state );
    t += dt;
}
```





FIXED DELTA TIME

- You don't know the display refresh rate ahead of time
- Your simulation will run faster or slower than you intended



VARIABLE DELTA TIME

Let's fix it! We can measure how long the previous frame takes, then pass that value as the delta time for the next frame.

```
double t = 0.0;
double currentTime = get_time_in_seconds();

while ( !quit )
{
    double newTime = get_time_in_seconds();
    double frameTime = newTime - currentTime;
    currentTime = newTime;

    integrate( state, t, frameTime );
    t += frameTime;

    render( state );
}
```





VARIABLE DELTA TIME

- Your game having a slightly different “feel” depending on framerate



SEMI-FIXED TIMESTEP

```
double t = 0.0;
double dt = 1 / 60.0;
double currentTime = get_time_in_seconds();

while ( !quit )
{
    double newTime = get_time_in_seconds(); double
    frameTime = newTime - currentTime;    currentTime =
    newTime;

    while ( frameTime > 0.0 )
    {
        float deltaTime = min( frameTime, dt );
        integrate( state, t, deltaTime );
        frameTime -= deltaTime;
        t += deltaTime;
    }

    render( state );
}
```





SEMI-FIXED TIMESTEP

- We're now taking multiple steps per-display update including one additional step to consume any the remainder of frame time not divisible by Δt . If your simulation is the most expensive part of your frame you could run into the so called "spiral of death"
- The simulation is not fully deterministic, because the last step has a variable timestep (the remainder)



DECOUPLED SIMULATION

**FIXED DELTA TIME
FOR THE SIMULATION**

**RENDER AT
DIFFERENT FRAMERATES**





DECOUPLED SIMULATION

```
double t = 0.0;
const double dt = 0.01;

double currentTime = get_time_in_seconds();
double accumulator = 0.0;

while ( !quit )
{
    double newTime = get_time_in_seconds();
    double frameTime = newTime - currentTime;
    currentTime = newTime;
    accumulator += frameTime;

    while ( accumulator >= dt )
    {
        integrate( state, t, dt );
        accumulator -= dt;
        t += dt;
    }

    render( state );
}
```





DECOUPLED SIMULATION

In the common case we have some unsimulated time left over at the end of each frame. This left over time is passed on to the next frame via the accumulator variable and is not discarded.

...but is it correct?





DECOUPLED SIMULATION

To understand what is going on consider a situation where the display framerate is 60fps and the physics is running at 50fps. There is no nice multiple, so the accumulator causes the simulation to alternate between mostly taking one and occasionally two physics steps per-frame when the remainders “accumulate” above dt .

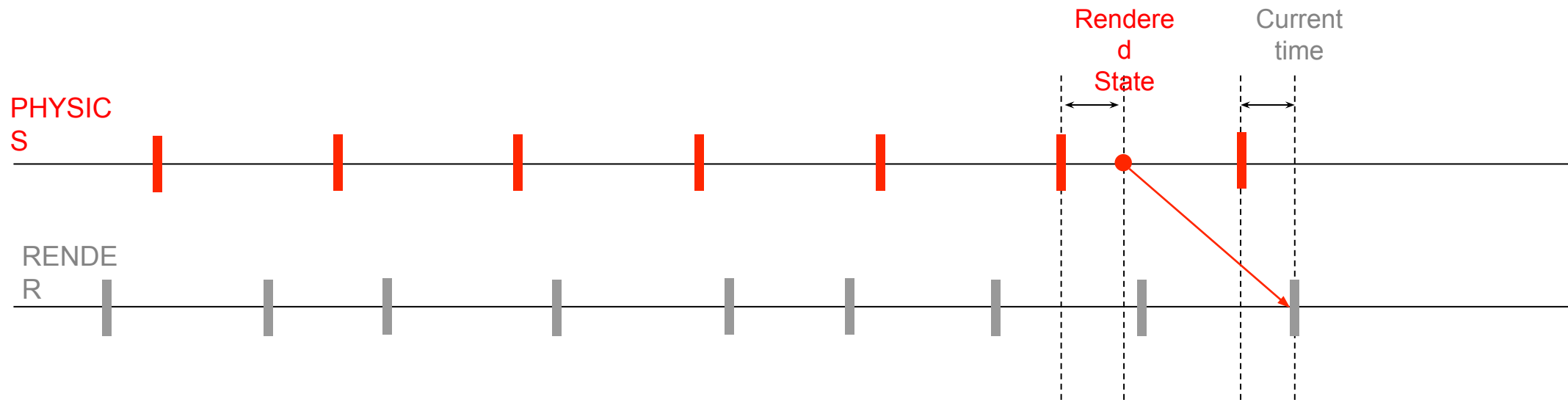
We are rendering the state of the physics simulation at a time different from the render time, and this can result in visual stuttering.



DECOUPLED SIMULATION

Let's fix it! We can interpolate between the previous and current physics state based on how much time is left in the accumulator.

We will be always “one frame behind”, but it's not noticeable. It's a very smart solution!





DECOUPLED SIMULATION

```
double t = 0.0;
double dt = 0.01;
double currentTime = get_time_in_seconds();
double accumulator = 0.0;

State previous;
State current;

while ( !quit )
{
    double newTime = get_time_in_seconds();
    double frameTime = newTime - currentTime;

    if ( frameTime > 0.25 ) frameTime = 0.25;

    currentTime = newTime;
    accumulator += frameTime;

    while ( accumulator >= dt )
    {
        previousState = currentState;
        integrate( currentState, t, dt );
        t += dt;
        accumulator -= dt;
    }

    const double alpha = accumulator / dt;

    State state = currentState * alpha + previousState * ( 1.0 - alpha );
    render( state );
}
```





REFERENCES

Real Time Collision Detection (*Christer Ericson*)

Game Physics Engine Development (*Ian Millington*)



That's all Folks!

TIME FOR QUESTIONS!

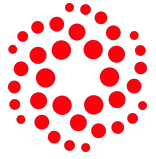




PHYSICS IN UNREAL ENGINE

“With great power comes great responsibility”

(Uncle Ben)



UNREAL PHYSICS

Now we have all the concepts we need, so, let's try!

